Parcel Decision Desk

Intern Challenge · Project Parcel · Synthetic prototype report

Scope warning. All shipments, carriers, updates, and results are generated. Day 2/3 updates are simulated, not official interviewer updates. This report is not evidence of real-company performance.

1. Problem as I understood it

At the 06:00 Asia/Kolkata dispatch snapshot, estimate whether each parcel will arrive after its **original customer promise**, then recommend an intervention only when assumed expected benefit exceeds cost and hard capacity allows it. The operations head cannot flag everything; the system must be selective, explainable, and honest about uncertainty.

Assumptions: backlog is a true as-of count; carrier contracts are effective-dated; intervention cost is Rs. 60–Rs. 110; avoided late cost is Rs. 450; intervention success is 40%. Efficacy and avoided cost are placeholders, not measured causal effects.

2. Approach and key decisions

Data. Deterministic generator plants duplicates, missingness, impossible weights, lb/kg mixing, aliases, mixed timezone offsets, and post-outcome fields. Audit normalizes units/aliases/timezones, recomputes the label from the original promise, and quarantines conflicting identities. Live inference rejects missing required values.

Leakage control. Feature allowlist excludes delivery time, final status, exception code, rating, actual cost, and revised promise. Chronological fit/calibration/selection/test split; rows whose outcomes are immature at the next boundary are purged. The Day 2 update is the forward test window.

Models. Smoothed carrier×service historical-rate baseline; regularized logistic regression; histogram gradient boosting. Both learned models are sigmoid-calibrated on a dedicated window and selected by expected net benefit on a separate window. Logistic regression was selected.

Decision. Expected net benefit = p × 0.40 × Rs. 450 – carrier cost. Positive-benefit parcels are greedily allocated under 80/day total, 25/day/carrier, and warehouse caps 35/30/25. Greedy is auditable but not proven globally optimal.

Rejected alternatives: random split (leaks future context), deep/AutoML complexity (harder to defend), automatic carrier write-back (unsafe without approval/auth), and revising the promise label (can hide lateness).

3. Results, with honest uncertainty

Forward test metric	Selected model	Same-window baseline
n / late rate	4,281 / 30.0%	4,281 / 30.0%
ROC AUC	0.683	0.626
PR AUC	0.471	0.385
Brier	0.202	0.204
Flagged	57	0
Precision at capacity	63.2%	0.0%
Recall at capacity	2.8%	0.0%
Expected benefit/day	Rs. 23	Rs. 0

Whole-day bootstrap 95% interval for recall at capacity: 1.9%–3.8%. This quantifies synthetic sampling variability, not real-world uncertainty.

Interpretation. The model ranks better than the baseline and is often right when it flags, but it flags very few late parcels because the economic threshold is strict. Estimated benefit is small. Economics, not capacity, is the binding constraint in this run.

Weakest segments

Segment	Value	n	Late rate	Recall	Brier
carrier_id	CAR_A	1,251	14.1%	0.0%	0.118
carrier_id	CAR_D	1,098	43.9%	0.0%	0.287
service_level	express	932	21.0%	0.0%	0.165
destination_region	SOUTH	863	28.5%	0.4%	0.200
parcel_type	fragile	596	29.0%	1.2%	0.201
destination_region	CENTRAL	671	28.9%	2.1%	0.194

The new carrier CAR_D has no pre-update training history and zero recall at capacity in the test. Express service and CAR_A also show low recall under the economic gate. These are measured failures, not guesses.

Global reliance

Selection-window permutation importance (mean ROC AUC drop): carrier_id (0.116), distance_km (0.054), service_level (0.030), warehouse_utilization (0.009), parcel_type (0.007). Carrier identity dominates. I trust this as an association in the synthetic generator, not as a causal claim or proof of real-world stability.

4. What failed and what I learned

F-001 Installation. System Python refused pip under PEP 668. Fixed by using a project-local virtual environment; README now starts there.

F-002 Demo export. Historical missing values entered the serving demo and were correctly rejected. Fixed the exporter to emit complete cases rather than weakening live validation.

F-003 Contract join. Nullable-string vs object dtype broke merge_asof. Added an explicit dtype boundary and an integration test through validation → contract join → prediction.

F-004 Monitoring boundary. Drift checks received raw CSV timestamp strings. Validated before monitoring and added a regression test.

F-005 First test run. API detail passed CSV timestamp strings into feature engineering; audit assumed an optional on-time flag existed. Fixed both. Lesson: a passing demo is not a passing contract.

5. Risks, limitations, and two more weeks

Risk	Current mitigation	Remaining gap
Synthetic-only evidence	Explicit labels and simulated updates	Replace with official data and re-run
Low recall / small benefit	Positive-benefit gate; shadow-pilot memo	Measure real intervention efficacy and cost
New carrier / drift	Unseen-category alarm; segment report	Manual-review policy and carrier-specific calibration
Leakage / optimism	Allowlist, chronological split, maturity purge	Production point-in-time backtest
Explanation overclaim	Local sensitivity caveat; global permutation	Human review and causal experiment

With two more weeks: (1) ingest the official extract and write a real point-in-time backtest; (2) run a small approved intervention experiment to estimate efficacy; (3) add carrier-specific calibration and a manual-review path for unseen carriers; (4) replace greedy allocation with an exact constrained optimizer if capacity becomes binding; (5) add auth, audit logs, and a persistent approval queue.

Recommendation

Run a two-week shadow pilot only. Keep human approval before customer contact or carrier changes. Watch daily input drift, weekly post-outcome calibration/recall, carrier-level precision/recall, intervention cost, and realized late rate. Do not broadly launch on synthetic evidence.

6. How to run everything

Python 3.11+. From a clean checkout:

Step	Command
Install	python -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]"
End-to-end demo	parcel demo
Dashboard/API	parcel serve --artifacts artifacts --host 0.0.0.0 --port 8000
Predict	parcel predict --input artifacts/demo_input.csv --output artifacts/predictions.csv
Drift check	parcel monitor --input artifacts/demo_input.csv # exit 2 = warning
Tests / lint	pytest && ruff check src tests
Rebuild report	python scripts/build_report.py

Deliverables map

Requirement	Where
Working system	src/parcel CLI + FastAPI + dashboard
Audit / leakage decisions	artifacts/DATA_AUDIT.md, artifacts/audit.json
Evaluation / error analysis	docs/EVALUATION.md, artifacts/metrics.json
Failure log	FAILURES.md
Daily check-ins	docs/checkins/day-1.md … day-3.md (drafts, not sent)
Decision memo	docs/DECISION_MEMO.md
Walkthrough	docs/WALKTHROUGH.md (script; record or present live)
AI disclosure / hours	docs/AI_USE.md, HOURS.md (human hours not fabricated)

Current artifact snapshot: 20,854 input rows, 20,696 clean, 55 quarantined; demo drift status warn. Tests: 26 passing at build time. Generated by scripts/build_report.py from artifacts; no metrics are hand-entered.